



Analysis & Development of Algorithms

IV

Anton Kovantsev

https://t.me/pop_antonij

Monte Carlo and meta-heuristic methods **VITMO**

Problem: $f: \mathbb{R}^n \rightarrow \mathbb{R}$; $f = f(x)$, where $x = (x_1, x_2 \dots x_n)$ is a row-vector. To solve the optimization problem $f(x) \rightarrow \min_{x \in Q}$ means to find $x^* \in Q$, where Q is the region of acceptability, such that f reaches a minimal value at x^* . $x^* = \operatorname{argmin}_{x \in Q} f(x)$. The approximation error is $\varepsilon > 0$. In the iterative algorithms below, we stop if $\|a_n - a_{n-1}\| < \varepsilon$, supposing, say, $x^* \approx (a_n + a_{n-1})/2$ with error ε .

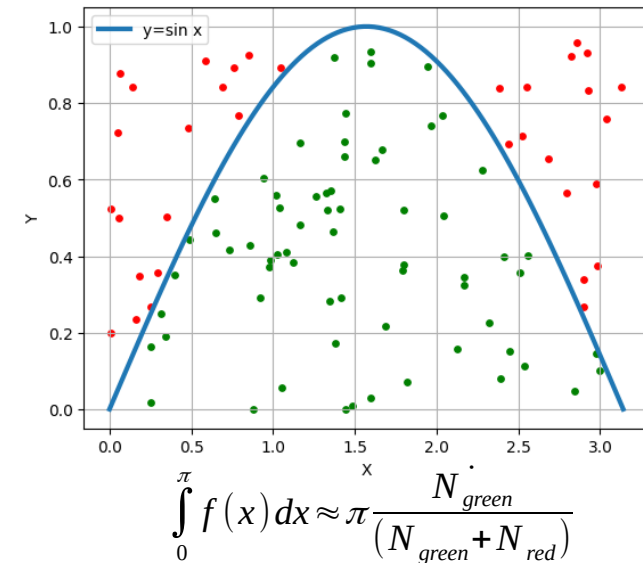
Monte Carlo (stochastic) methods are a wide class of algorithms that rely on repeated random sampling to obtain solutions to an optimization problem. The methods are most useful when it is impossible or difficult to apply other approaches.

Probabilistic interpretation: by the law of large numbers, the expected value of some random variable can be approximated by taking the empirical mean of independent samples of the variable.

Metaheuristic methods are nature-inspired algorithms that discover a solution to an optimization problem by trial and error. Metaheuristics do not guarantee that a globally optimal solution can be found. Many metaheuristics implement some form of stochastic optimization.

"I, at any rate, am convinced that God does not throw dice", once Einstein said.

"Einstein, don't tell God what to do", Niels Bohr remarked.

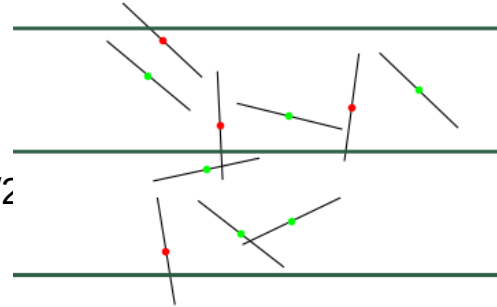


Buffon's needle problem

Given a needle of length l dropped on a plane ruled with parallel lines t units apart, what is the probability P that the needle will lie across a line upon landing? Suppose that $l < t$ ("short needle" case).

Probability depends on:
Needle length l ;
Lines density $1/t$;
Angle between a needle and a line – uniform in $[0, \pi/2]$

$$P = \frac{2l}{\pi t}$$

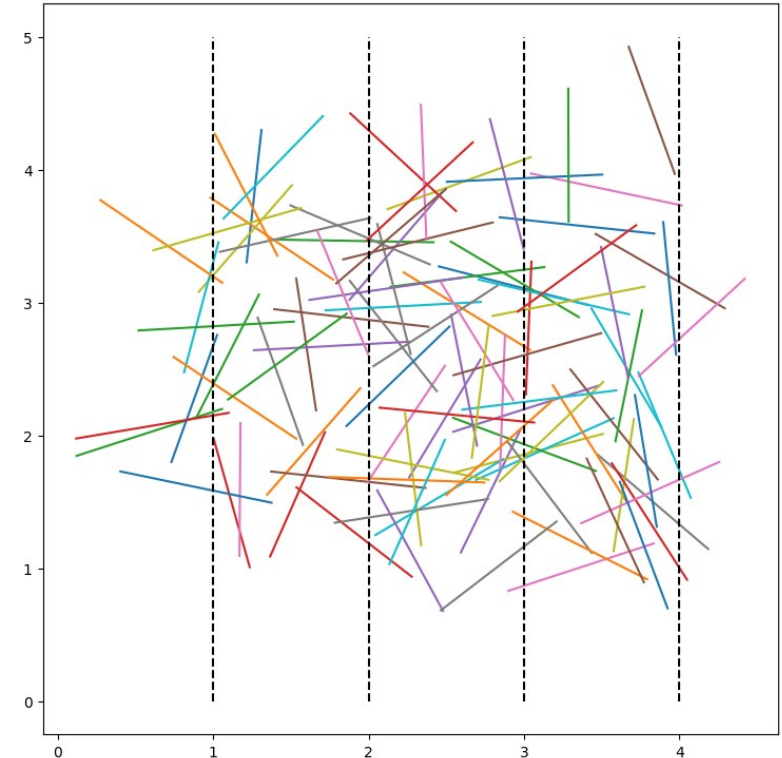


Drop n needles and find that m of those are crossing lines, so P is approximated by the fraction m/n . From this, find an experimental estimate for π :

$$\pi = \frac{m}{n} \frac{2l}{t}$$

Buffon's Needle Simulation with 100 samples

Estimated value of pi: 3.1746031746031744, Actual Value of pi: 3.141592653589793, Diff = 0.03301052101338131



Simulated annealing

Simulated annealing (Metropolis algorithm) is a metaheuristic algorithm that solves the optimization problem similar to the process of annealing in metallurgy.

Name and inspiration: heating and controlled cooling of a material to increase the size of its crystals and reduce their defects.

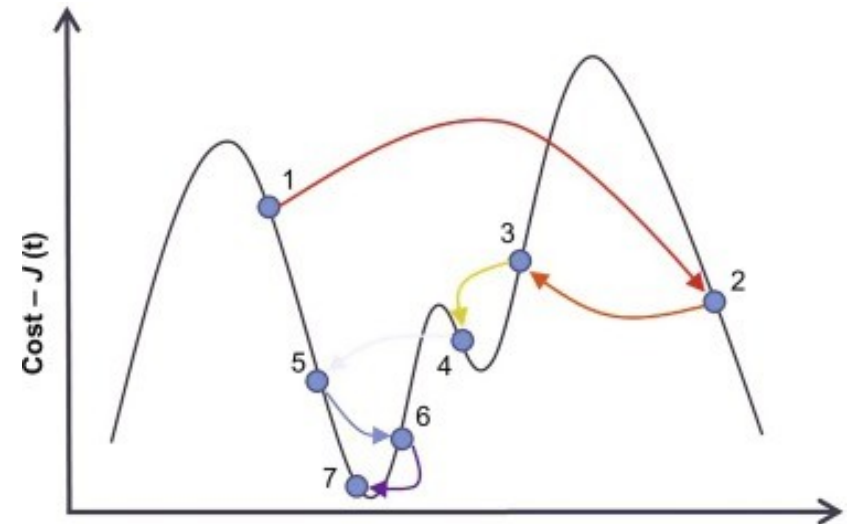
Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be energy. Let $T = \{T_k\}$ be a decreasing non-negative sequence s.t. $T_k = 0$ for $k > N$ (T is called cooling schedule or temperature).

The goal is to bring the system, from an arbitrary initial state, to a state with the minimum possible energy.

Let a_0 be an initial approximation. At each iteration $k \in \mathbb{N}_0$:
choose $a^* \in \text{Neighbours}(a_k)$, where Neighbours is a certain rule;
if $f(a^*) \leq f(a_k)$, then $a_{k+1} = a^*$
else if $f(a^*) > f(a_k)$, then $a^* = a_k$ with probability

$$\exp\left(\frac{-f(a^*) - f(a_k)}{T_k}\right)$$

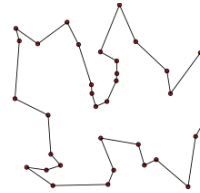
Stop when $T_k = 0$



Travelling Salesman Problem

TSP formulation:

Given a list of cities and their locations (usually specified as points on a plane), what is the shortest route which will visit every city exactly once and return to the point of origin.



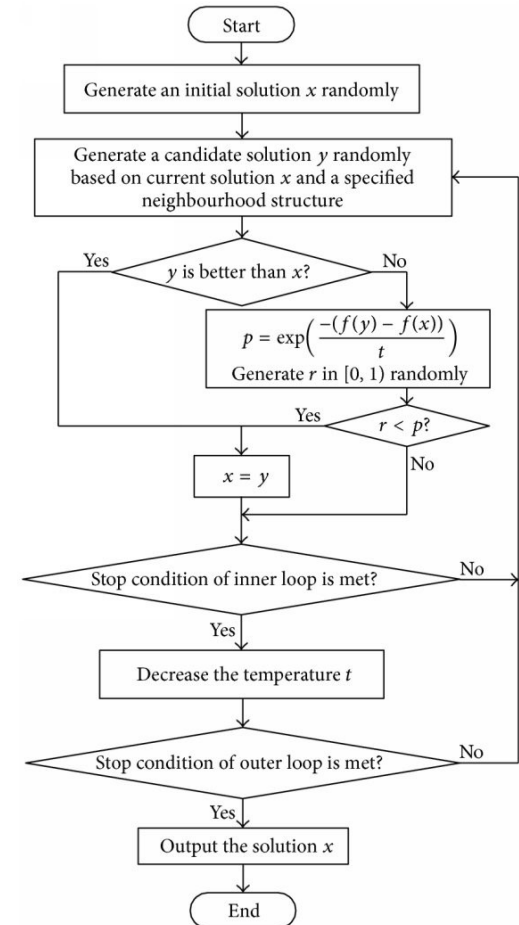
- 1) Create the initial list of cities by shuffling the input list (ie: make the order of visit random).
- 2) At every iteration, two cities are swapped in the list. The cost value is the distance traveled by the salesman for the whole tour.
- 3) If the new distance, computed after the change, is shorter than the current distance, it is kept.
- 4) If the new distance is longer than the current one, it is kept with a certain probability.
- 5) We update the temperature at every iteration by slowly cooling down.

Two major optimizations can be used to speed up the computation of the distances:

A) The distances between all pairs of cities can be precomputed in a table, and used thereafter. A triangular matrix is enough.

B) As a change in parameters consists in swapping two cities, it is useless to recompute the total distance for the whole tour. Indeed, only the distances changed by the swap should be recomputed.

You know how to handle it: <https://www.youtube.com/watch?v=NPE3zncXA5s>
<https://codecapsule.com/2010/04/06/simulated-annealing-traveling-salesman/>

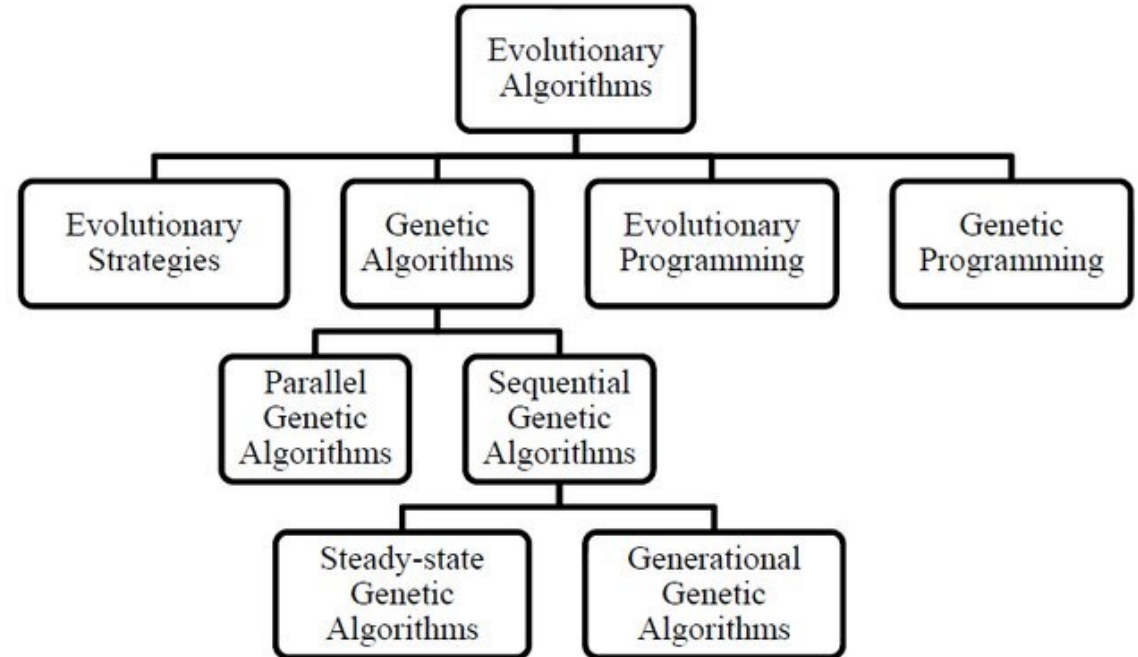
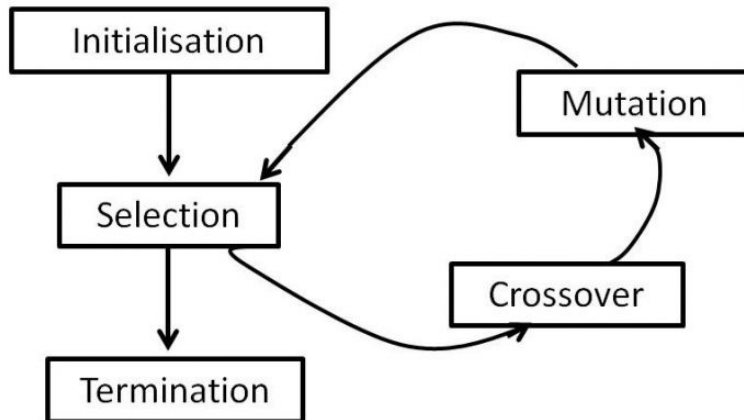


Evolutionary algorithms

Evolutionary algorithms are generic population-based metaheuristic optimization algorithms. They use mechanisms inspired by biological evolution, such as reproduction, mutation, recombination, and selection. Candidate solutions to the optimization problem play the role of individuals in a population, and the function under optimization determines the quality of the solutions. Evolution of the population then takes place after the repeated application of the above-mentioned mechanisms.

Examples:

- Differential evolution
- Particle swarm optimization
- Genetic algorithm (self-study)



Differential evolution

Differential evolution is an metaheuristic algorithm that solves the optimization problem by maintaining a population of agents, i.e. candidate solutions, creating new agents by combining existing ones and further keeping the best one.

Choose $p \in [0, 1]$, the crossover probability, $w \in [0, 2]$, the differential weight, and $N \geq 4$, the population size. Let $x \in \mathbb{R}^n$ denote an agent in the population.

Until a termination criterion is met (e.g. the number of iterations performed):

Randomly pick N agents x (i.e. the population).

Pick three distinct agents a , b and c from the population, different from x .

Compute the trial vector $y = (y_1, \dots, y_n)$ as follows.

For $i = 1, \dots, n$, pick $r_i \in U(0, 1)$. If $r_i < p$, then $y_i = a_i + w (b_i - c_i)$, otherwise $y_i = x_i$.

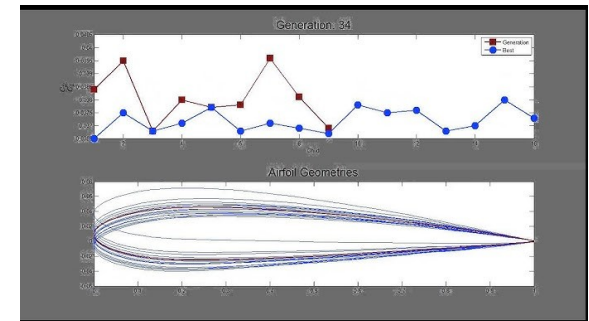
If $f(y) \leq f(x)$, then replace x with the trial vector y , otherwise keep x .

Pick the best agent from the population and return it as the best found solution.

Applications:

Airfoil geometry <https://www.youtube.com/watch?v=vzcrW6xZAtI>

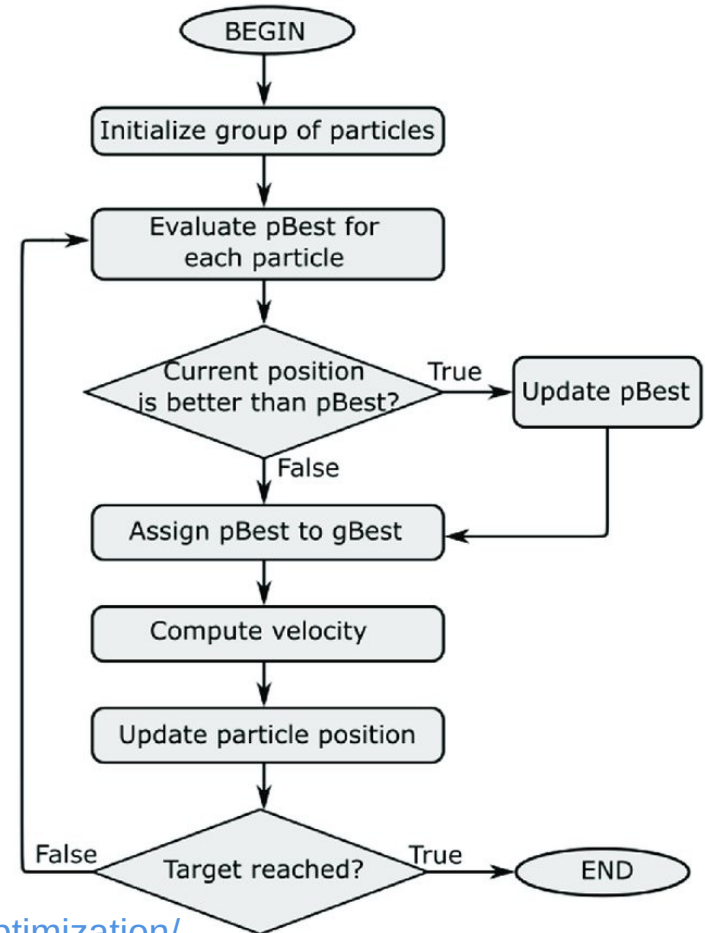
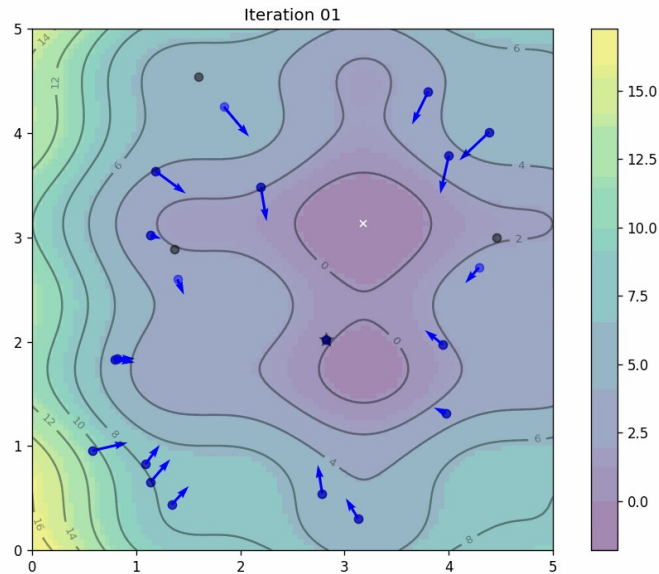
Generative design



Particle swarm optimization

Particle swarm optimization is a metaheuristic algorithm that solves the optimization problem by iterative moving candidate solutions (particles) with certain velocity. Each particle's movement is influenced by its local best known position and is guided by the swarm's best known position that is updated as better positions are found by other particles.

<https://www.youtube.com/watch?v=7uZcuaUvwq0>



<https://machinelearningmastery.com/a-gentle-introduction-to-particle-swarm-optimization/>

Particle swarm optimization

Let s be the number of particles in the swarm, with positions x_i and velocities v_i .
Let p_i be the best known position of particle i and g the one of the entire swarm.

All the vectors are d -dimensional

The values b_l and b_u represents the lower and upper boundaries of the search-space. The parameters ω , φ_p , and φ_g are selected manually and control the behaviour and efficiency of the method. The choice of parameters can have a large impact on optimization performance and is the subject of much research.

Initialize the swarm's best known position g with a random number

For each particle $i = 1, \dots, s$ do:

Initialize the particle's position with a uniformly distributed $x_i \sim U(b_l, b_u)$

Initialize the particle's best known position to its initial position: $p_i \leftarrow x_i$

If $f(p_i) < f(g)$ then update the swarm's best known position: $g \leftarrow p_i$

Initialize the particle's velocity: $v_i \sim U(-|b_u - b_l|, |b_u - b_l|)$

Until a termination criterion is met (e.g. a certain number of iterations is reached):

For each particle $i = 1, \dots, s$ do

For each $d = 1, \dots, n$ do

Pick random numbers: $r_p, r_g \sim U(0, 1)$

Update the velocity: $v_{i,d} \leftarrow \omega v_{i,d} + \varphi_p r_p (p_{i,d} - x_{i,d}) + \varphi_g r_g (g_d - x_{i,d})$

Update the position: $x_i \leftarrow x_i + v_i$

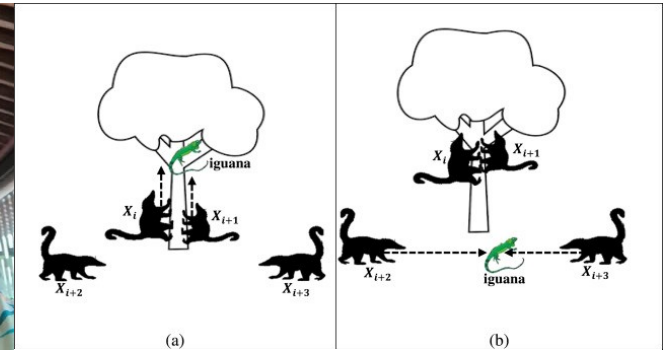
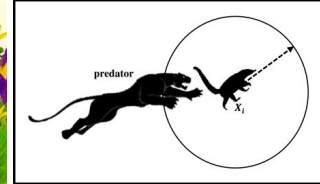
if $f(x_i) < f(p_i)$ then update the particle's best known position: $p_i \leftarrow x_i$

if $f(p_i) < f(g)$ then update the swarm's best known position: $g \leftarrow p_i$

Optimization Zoo: pyMetaheuristic

Ant Lion Optimizer
 Artificial Bee Colony Optimization
 Artificial Fish Swarm Algorithm
 Bat Algorithm
 Biogeography Based Optimization
 Cat Swarm Optimization
 Chicken Swarm Optimization
 Clonal Selection Algorithm
 Coati Optimization Algorithm
 Cockroach Swarm Optimization
 Crow Search Algorithm
 Cuckoo Search
 Cultural Algorithm
 Dragonfly Algorithm
 Elephant Herding Optimization Algorithm
 Firefly Algorithm
 Flower Pollination Algorithm
 Genetic Algorithm
 Grey Wolf Optimizer
 Grasshopper Optimization Algorithm
 Harris Hawks Optimization
 Hunting Search Algorithm
 Jaya
 Jellyfish Search Optimizer
 Krill Herd Algorithm
 Monarch Butterfly Optimization
 Moth Flame Optimization
 Population-Based Incremental Learning
 Salp Swarm Algorithm
Student Psychology Based Optimization
 Symbiotic Organisms Search
 Teaching Learning Based Optimization
 Whale Optimization Algorithm

<https://github.com/Valdecy/pyMetaheuristic> by Valdecy Pereira



$$Iguana^G : Iguana_j^G = lb_j + r \cdot (ub_j - lb_j), \quad j = 1, 2, \dots, m, \quad (5)$$

$$X_i^{P1} : x_{i,j}^{P1} = \begin{cases} x_{i,j} + r \cdot (Iguana_j^G - I \cdot x_{i,j}), & F_{Iguana^G} < F_i, \\ x_{i,j} + r \cdot (x_{i,j} - Iguana_j^G), & \text{else,} \end{cases}$$

for $i = \lfloor \frac{N}{2} \rfloor + 1, \lfloor \frac{N}{2} \rfloor + 2, \dots, N$ and $j = 1, 2, \dots, m$.

About Coati in details:

<https://www.sciencedirect.com/science/article/pii/S09507051220110427>

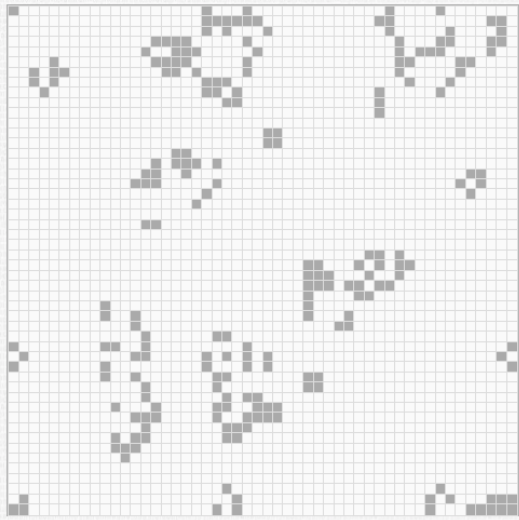
Agent base modelling

The **Game of Life** is a cellular automaton invented by John Conway in 1970.

Rules:

- a dead cell becomes alive if there are exactly three living cells next to it;
- if a living cell is surrounded by two or three living cells, then this cell remains alive;
- otherwise the cell becomes dead.

<https://programforyou.ru/projects/life>



Flocking behaviour (Murmuration) (Latin murmuratio - "muttering, buzzing, croaking")

The basic patterns of flock behaviour are controlled by three simple rules:

- Separation - Avoid crowding of neighbours (short-range repulsion)
- Alignment - Direct towards the average direction of neighbours
- Cohesion - Direct towards the average position of neighbours (long-range attraction)

With these three simple rules, the flock moves extremely realistically, creating complex movements and interactions that would be extremely difficult to create in any other way.

https://agentpy.readthedocs.io/en/latest/agentpy_flocking.htm



Agent base modelling

For crowd modeling:

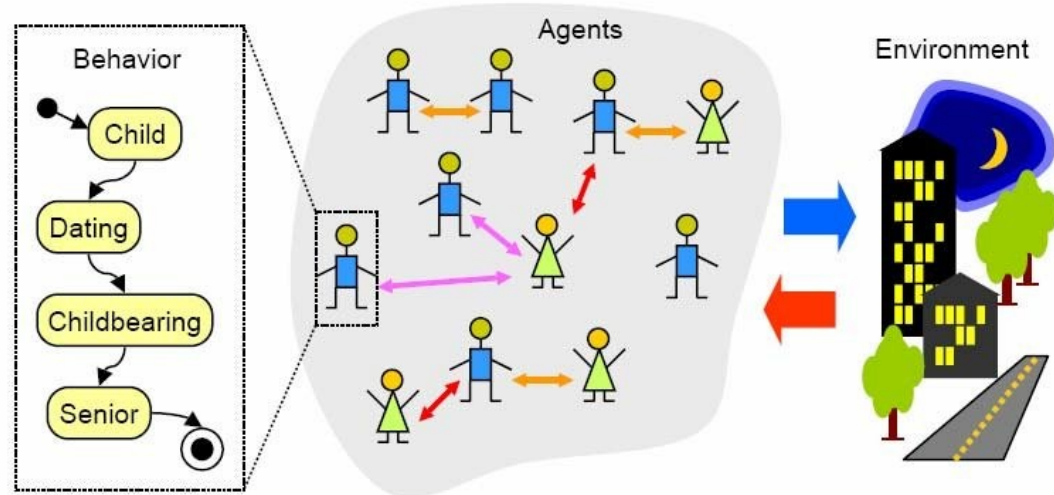
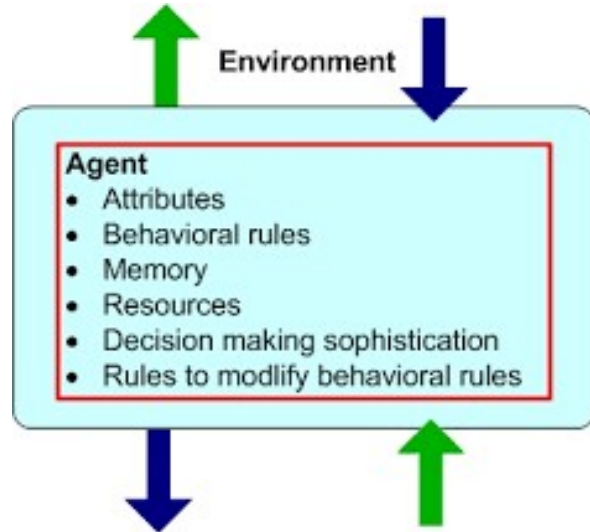
The strategic component is the top level of the agent's decision making. It determines where and when the agent will go by assigning it goals.

The tactical component finds a route to the goal and sets the agent's preferred speed.

The operational component calculates the final speed taking into account neighbors, obstacles, and the preferred speed.

The environment model defines the conditions for the existence and activity of agents.

The simulator is responsible for the modeling process and the execution flow, and also stores other important components, such as the list of all agents, all models, all groups, and the navigation system.





АНТОН КОВАНЦЕВ
pop.antonij@gmail.com
https://t.me/pop_antonij